Experiment - 6

Error Detection & correction using Hamming Code

Aim:

To write a program to implement error detection & correction using the Hamming code.

Algorithm:

1) Read the binary data bits
2) Count the number of data bits m.
3) Find the number of parity bits r using

$$2^r \geq m+r+1$$

4) Create position for Hamming Code
5) Place parity bits at postion 1, 2, 4, 8...
6) Calculate each parity bit using XOR
7) Display the Hamming Code
8) Read the received Hamming code
9) check the parity bits again
10) Read the position of the parity bits that fail to find the errors position.
11) If the error position is not zero, flip the bit to correct the error.
12) Display the Hamming code

```
import java.util.Scanner;

public class Hamming code{

    static int findparityBits (int m)
        int r=0;

        while (Math.pow (2,r) <m+r+1){
            r++;
        }
        return r;

    Static string encode (string data){
        int m = data.length();
        int r= findParityBits (m);
        int n = m+r;

    int[] code = new int [n+1];
        int j=0

    for (int i=1; i<n; i++){
        if((i & (i-1)) !=0){
            code[i] = data.charAt(j)-'0';
            j++;
        }
    }

for (int p=0; p<r; p++){

    int pos = (int) Math.pow(2,p);
        int parity = 0;
    for (int i=1; i<n; i++){
        if ((i & pos) !=0){
            parity = parity ^ code[i];
    }
}
code[pos] = parity;
}
```

```
        String result = "";
        for (int i = 1; i <= n; i++){
            result += code[i];
        }
        return result;
    }

static String correctErrorString(String received){
    int n = received.length();
    int[] code = new int[n+1];

    for (int i = 1; i <= n; i++){
        code[i] = received.charAt(i-1) - '0';
    }

    int x = 0;

    while (Math.pow(2,x) < (n+1)){
        x++;
    }

    int errorposition = 0;

    for (int p = 0; p < x; p++){
        int pos = (int) Math.pow(2,p);
        int parity = 0;

        for (int i = 1; i <= n; i++){
            if((i & position) != 0){
                parity = parity + code[i];
            }
        }

        if (parity != 0){
            errorPart = position;
        }
    }
```

```
if (errorPos == 0){
    System.out.println("No error detected");
}
else{
    System.out.println("Error detected at pos:"
                        + errorPos);
    if (code[errorPos] == 0){
        code[errorPos] = 1;
    }
    else{
        code[errorPos] = 0;
    }
    System.out.println("Error corrected");
}
String corrected = "";
for (int i = 1; i <= n; i++){
        corrected += code[i];
    }
    return corrected;
}
public static void main(String[] args){
    Scanner sc = new Scanner(System.in)
    System.out.println("Enter data bit:");
        String data = sc.nextLine();
    String hammingCode = encode(data);
    System.out.println("Generated Hamming
                        Code:" + hammingcode);
    System.out.println("Enter received code:");
        String received = sc.nextLine();
    String corrected = correctError(received);
```

```
        System.out.println("Corrected Hamming code: " +
                                corrected);
        sc.close();
    }
}
```

Output:-

```
Enter data bits: 1011
Generated Hamming Code: 0110011
Enter received code: 0110111
Error detected at position: 5
Error corrected.
Corrected Hamming Code: 0110011
```

Result:-

The Hamming code program was successfully implemented in Java.